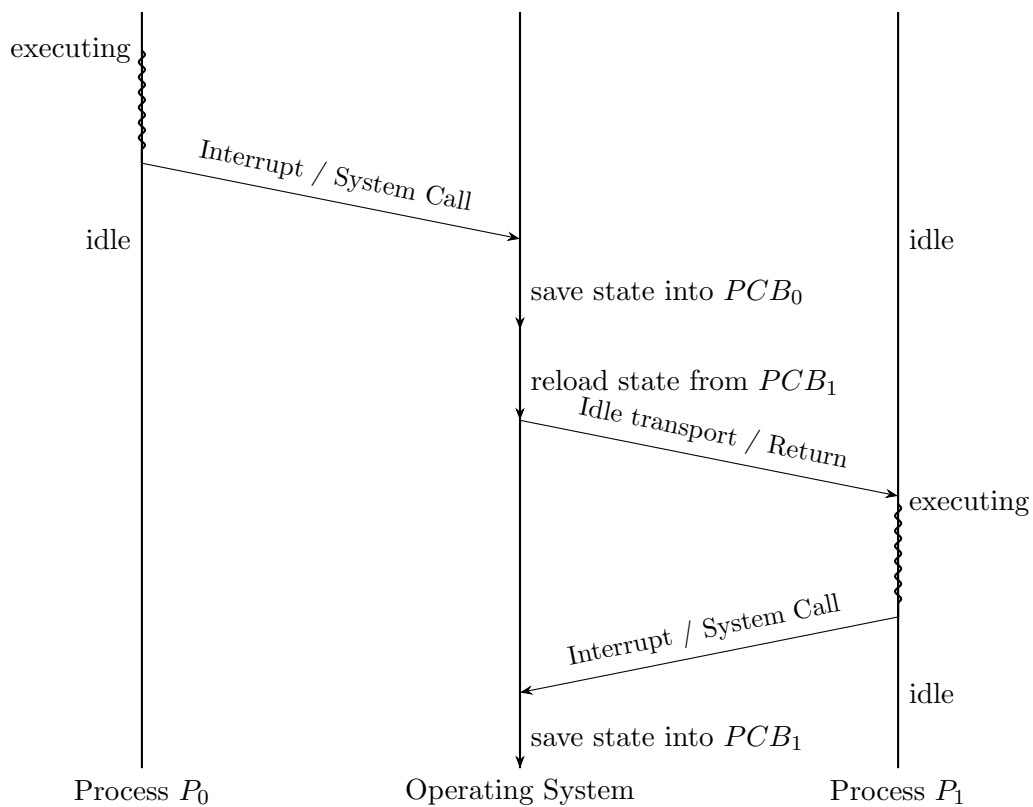


GHI CHÉP HỆ ĐIỀU HÀNH (OPERATING SYSTEMS)

1. Sơ đồ Context Switch (Chuyển ngữ cảnh)



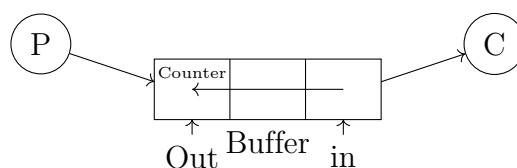
3. Thao tác trên tiến trình

- Tạo tiến trình.
- Kết thúc tiến trình.

4. Phân loại tiến trình (không ghi chi tiết)

5. Liên lạc giữa các tiến trình

- **Bài toán:** Người sản xuất (Producer) - Người tiêu thụ (Consumer).



- Hệ thống gồm 2 tiến trình.

- Producer tạo ra các sản phẩm.
- Consumer tiêu thụ các sản phẩm được tạo ra.
- Producer và Consumer hoạt động đồng thời.
- Sử dụng vùng đệm dùng chung (buffer) chứa sản phẩm được điền vào bởi Producer và lấy ra bởi Consumer.
- **IN:** Vị trí trống kế tiếp trong vùng đệm.
- **OUT:** Vị trí dữ liệu đầu tiên trong vùng đệm.
- **Counter:** Số sản phẩm trong vùng đệm.

Liên lạc trực tiếp:

- Các tiến trình phải gọi tên tiến trình nhận/gửi một cách tường minh.
- `send(P, message)`: Gửi một thông báo tới tiến trình P.
- `receive(Q, message)`: Nhận một thông báo từ tiến trình Q.

Liên lạc gián tiếp:

- Các thông điệp được gửi/nhận tới/từ các hòm thư (mailboxes), cổng (ports).
- Các hòm thư có định danh duy nhất.
- Các tiến trình có thể trao đổi nếu chúng dùng chung hòm thư.
- *Các thao tác:*
 - Tạo hòm thư.
 - Gửi/nhận thông báo qua hòm thư.
 - `send(A, msg)`: gửi 1 msg tới hòm thư A.
 - `receive(A, msg)`: nhận msg từ A.
 - Hủy bỏ hòm thư.

Vấn đề đồng bộ hóa:

- Truyền thông điệp có thể chờ (blocking): từ tháng gửi hoặc chờ đợi (non-blocking).

Vùng đệm:

- Hàng đợi có thể cài đặt như sau:
- Khả năng chứa 0.
- Khả năng chứa có hạn (hữu hạn).
- Khả năng chứa không hạn (vô hạn).

Liên lạc trong Client - Server

2. Luồng (Thread)

- Là đơn vị sử dụng CPU cơ bản gồm:

- Định danh luồng (ID Thread).
- Bộ đếm chương trình (Program Counter).
- Tập các thanh ghi (Registers).
- Không gian Stack.
- Chia sẻ cùng các luồng khác trong cùng một tiến trình:
- Đoạn mã lệnh.
- Đoạn dữ liệu.
- Các tài nguyên HDH khác.

Phần 2: Quản lý tiến trình (tt)

3. Điều phối CPU

Nguyên tắc: Lựa chọn một trong số các tiến trình đang trong bộ nhớ và cung cấp CPU cho nó. Các tiến trình phải sắp hàng trong hàng đợi.

- Điều phối CPU xảy ra khi tiến trình:
 1. Chuyển từ trạng thái thực hiện sang trạng thái chờ đợi.
 2. Chuyển từ trạng thái thực hiện sang trạng thái sẵn sàng (ngắt thời gian/hết thời gian sử dụng CPU).
 3. Chuyển từ trạng thái chờ đợi sang trạng thái sẵn sàng (hoàn thành vào/ra).
 4. Tiến trình kết thúc.

Tiêu chuẩn điều phối I

- Sử dụng CPU lớn nhất.
- Thông lượng (throughput) lớn nhất.
- Thời gian hoàn thành nhỏ nhất.

Tiêu chuẩn điều phối II

- Thời gian chờ đợi (nhỏ nhất).
- Thời gian đáp ứng (nhỏ nhất).

Xét Ví dụ: Bài toán: Có 3 process P_1, P_2, P_3 với thời gian CPU cần thực hiện (Burst time) tương ứng là 24, 3, 5. Thời gian yêu cầu CPU thực hiện (Thời gian nhập/Arrival time) lần lượt là 0, 5, 7.

Process	T.thực hiện	T.nhập
P_1	24	0
P_2	3	5
P_3	5	7

Ta xem xét bài toán trên với các phương pháp lựa chọn tiến trình cụ thể:

- **FIFO / FCFS** (First In First Out / First Come First Served).
- **SJF** (Shortest Job First) → Có thể độc quyền hoặc không độc quyền.
- **RR** (Round Robin).
- **ĐQ** (Độc quyền).
- **SRT** (Shortest Remaining Time).
- **MLF** (Multi-Level Feedback).

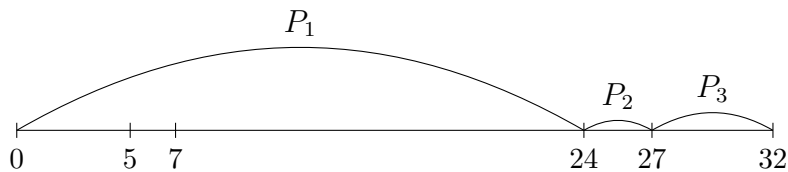
PP FIFO: Tiến trình nào đến trước phục vụ trước

Process	T.thực hiện	T.nhập	TKT	T.chờ	T.lưu
P_1	24	0	24	0	24
P_2	3	5	27	19	22
P_3	5	7	32	20	25

Công thức:

- $T.chờ = TKT - T.nhập - T.thực\ hiện$
- $T.lưu = TKT - T.nhập$

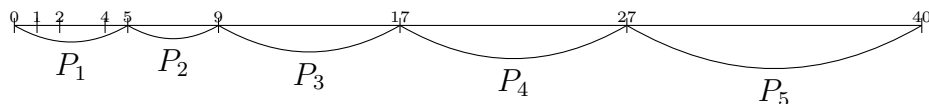
Biểu đồ Gantt (FIFO):



$$\sum T.ch = 39 \Rightarrow T.ch_{TB} = 13$$

Câu 24:

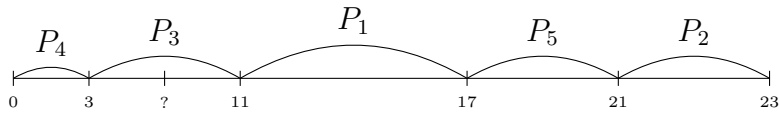
Process	T.nhập	T.thực hiện	TKT	T.chờ
P_1	0	5	5	0
P_2	1	4	9	4
P_3	2	8	17	7
P_4	4	10	27	13
P_5	9	13	40	18



$$\Rightarrow \sum T.ch = 42$$

Ví dụ khác:

Process	T.th	T.nhập	TKT	T.chờ	T.lưu
P_1	6	2	17	9	15
P_2	2	5	23	16	17
P_3	8	1	11	2	10
P_4	3	0	3	0	3
P_5	4	4	21	13	17



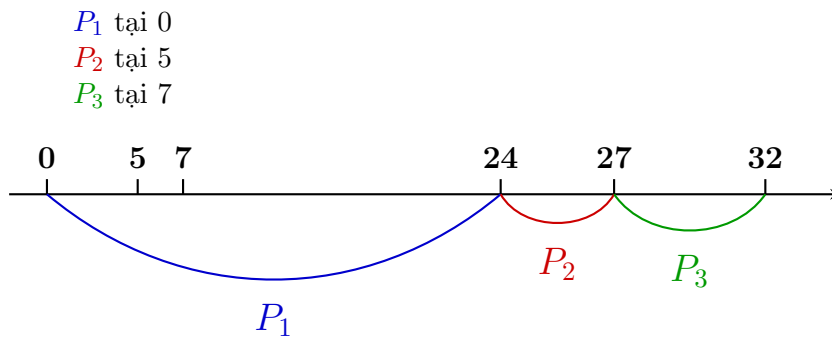
Phương pháp SJF (Shortest Job First)

SJF: Tiến trình nào có thời gian ngắn nhất thì thực hiện trước.

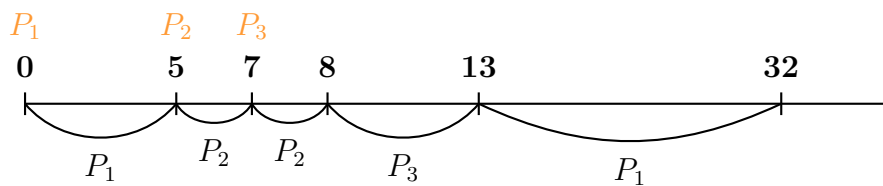
- **SJF Độc quyền (Non-preemptive):** Khi 1 tiến trình đang thực thi thì nó không bị dừng bởi các tiến trình khác dù tiến trình khác có thời gian thực thi ngắn hơn.
- **SJF Không độc quyền (Preemptive):** Một tiến trình đang thực thi vẫn bị dừng bởi các tiến trình có thời gian thực thi ngắn hơn.

SJF ĐQ (Độc quyền)

Process	T.th	T.nhập	TKT	T.chờ	T.lưu
P_1	24	0	24	0	24
P_2	3	5	27	19	22
P_3	5	7	32	20	25



SJF KĐQ (Không độc quyền - Shortest Remaining Time)



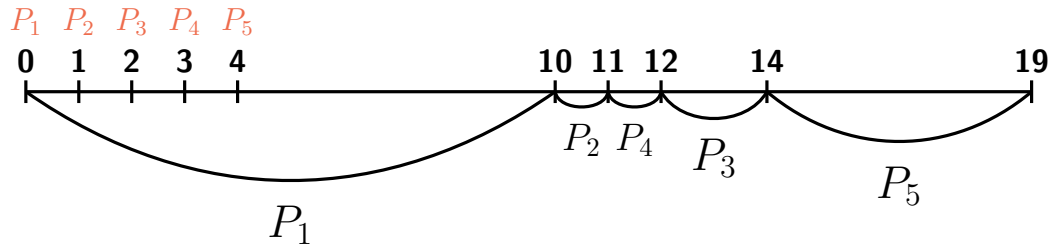
Process	T_{th}	T_{nhp}	T_{KT}	T_{ch}	T_{lu}
P_1	24	0	32	8	32
P_2	3	5	8	0	3
P_3	5	7	13	1	6

$$\sum T_{ch} = 9 \quad t_{ch}TB = 3$$

Bài tập:

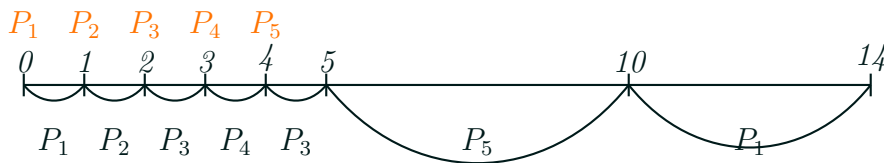
1. ĐQ (Độc quyền)

Process	T.th	T.nhập	TKT	T.chờ	T.lưu
P_1	10	0	10	0	10
P_2	1	1	11	9	10
P_3	2	2	14	10	12
P_4	1	3	12	8	9
P_5	5	4	19	10	15



2. KĐQ (Không độc quyền)

Process	T.th	T.nhập	TKT	T.chờ	T.lưu
P_1	10	0	19	9	19
P_2	1	1	2	0	1
P_3	2	2	5	1	3
P_4	1	3	4	0	1
P_5	5	4	10	1	5



Phương pháp Round Robin

- Sử dụng 1 hàng chờ trật tự, process đến trước sẽ được phục vụ trước trong 1 khe thời gian nhất định (quantum time).
- Khi hết khe thời gian mà process vẫn cần chạy tiếp, nó sẽ tự động trở về cuối hàng đợi để chờ lượt phục vụ kế tiếp.

Ưu điểm:

- Thời gian chờ đợi ít.
- Đối với các quá trình liên quan đến nhập xuất I/O người dùng thì sẽ rất hiệu quả.
- Việc cài đặt không quá phức tạp.

Nhược điểm:

- Thời gian chờ trung bình dưới chính sách RR là quá dài.
- Nếu thời gian định mức cho việc xử lý quá lớn thì RR thành FIFO.
- Nếu thời gian quá ngắn so với thời gian xử lý của một tiến trình trong danh sách hàng đợi thì việc chuyển đổi ngữ cảnh xảy ra quá nhiều.

Phương pháp theo quyền ưu tiên

- Ý tưởng của pp này là mỗi tiến trình sẽ được gán 1 mức độ ưu tiên nhất định và đưa vào hàng chờ. Khi cần chọn tiến trình tiếp theo, hệ thống sẽ chọn tiến trình có mức ưu tiên cao nhất bất chấp nó đến trước hay sau.

- **Nguyên tắc:**

- Độc quyền.
- Không độc quyền.

Ví dụ mức ưu tiên $3 > 2 > 1$.

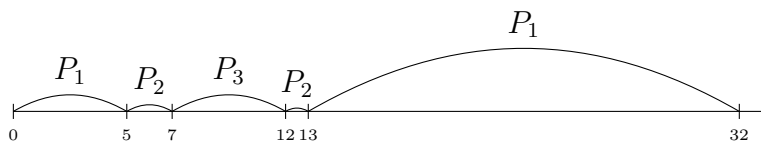
1. Độc quyền

P	T.th	T.nhập	Ưu tiên	TKT	T.chờ	T.lưu
P_1	24	0	1	24	24	24
P_2	3	5	2	32	27	27
P_3	5	7	3	29	22	22

(Lưu ý: Số liệu T.chờ trong hình có vẻ chưa trừ đúng T.nhập/T.thực hiện theo công thức chuẩn, đây là sao chép lại ghi chép)

2. Không độc quyền

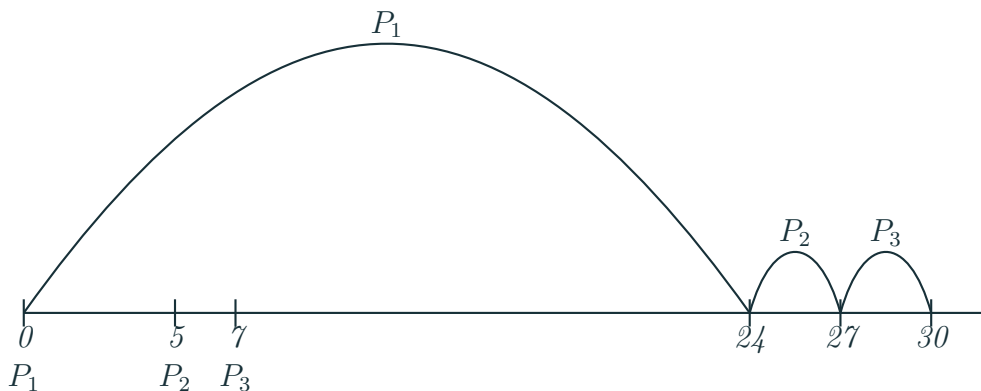
P	T.th	T.nhập	Ưu tiên	TKT	T.chờ	T.lưu
P_1	24	0	1	32	8	32
P_2	3	5	2	13	5	8
P_3	5	7	3	12	0	5



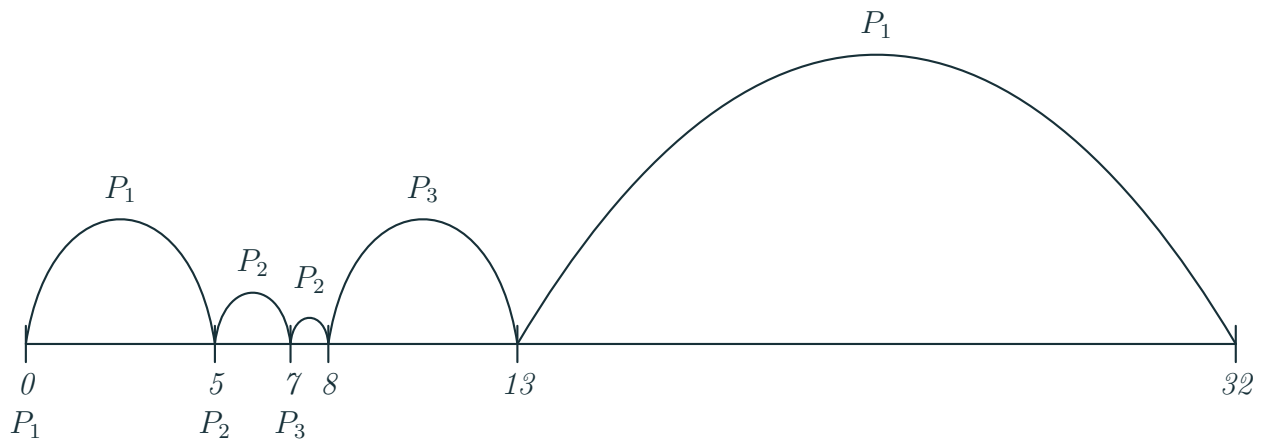
Phương pháp SRT (Shortest Remain Time)

- Tương tự như SJF nhưng trong thuật toán này, độ ưu tiên thực hiện các tiến trình dựa vào thời gian cần thiết để thực hiện nốt tiến trình.

Độc quyền



Ko độc quyền

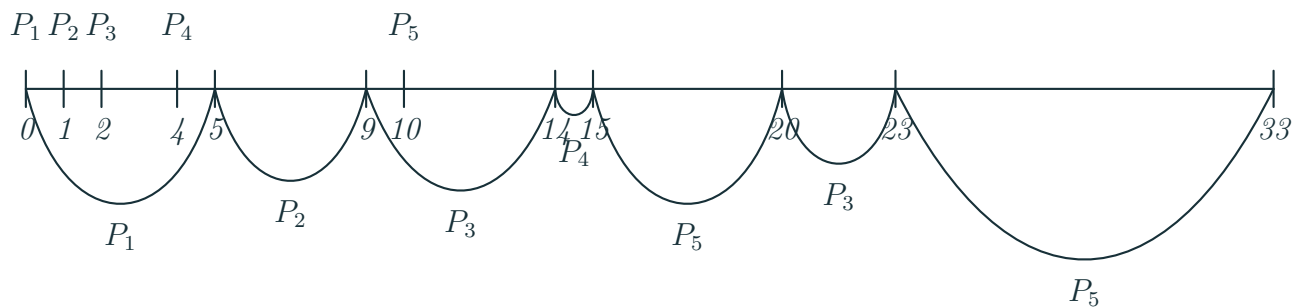


Multi-level Feedback

- Trong phương pháp sử dụng quyền ưu tiên, cái tiến trình có mức ưu tiên thấp rất dễ rơi vào tình trạng đói "CPU". Để khắc phục tình trạng này người ta xây dựng phương pháp sử dụng nhiều mức ưu tiên với ý tưởng sau mỗi lần sử dụng CPU mức ưu tiên của tiến trình sẽ giảm đi.

Bài tập Round Robin (R=5)

Process	T.th	T.nhập	...
1	5	0	
2	4	1	
3	8	2	
4	1	4	
5	15	10	



Phần II: Quản lý tiến trình (tiếp)

4. Tài nguyên căng và điều độ tiến trình

Qui ước:

- Có 2 tiến trình P_1 và P_2 thực hiện đồng thời.
- Các tiến trình dùng chung một tài nguyên căng.
- Mỗi tiến trình đặt đoạn căng ở đâu, tiếp theo là phần còn lại.
 - Tiến trình phải xin phép trước khi vào đoạn căng { phần vào }.
 - Tiến trình khi thoát khỏi đoạn căng thực hiện { phần ra }.

Cấu trúc tổng quát của một tiến trình:


```

Repeat
    Phần vào
    { Đoạn găng của tiến trình }
    Phần ra
    { Phần còn lại của tiến trình }
Until false

```

Các phương pháp giải quyết bài toán tới hạn

1. Phương pháp khóa trong

- Chỉ 1 tiến trình được vào đoạn tới hạn còn các tiến trình khác phải chờ.
- Sử dụng 1 byte khóa K để làm khóa. Khi TT vào tới hạn $K = 1$, khi ra khỏi đoạn tới hạn $K = 0$.
- Trước khi vào đoạn tới hạn các TT phải Kiểm tra.

Thuật toán Dekker

- **Ý tưởng:** Thuật toán DEKKER sử dụng thêm 1 biến trạng thái tt để xác định mức độ ưu tiên giữa các tiến trình cùng muốn vào đoạn găng.

```

Var c1, c2, tt: Integer;
BEGIN c1 := 0; c2 := 0; tt := 1;
PARBEGIN

```

```

TT1: Repeat
    c1 := 1;
    While c2 = 1 do
        if tt = 2 then
            begin c1 := 0;
                while tt = 2 do ;
                    c1 := 1
            end;
        { Doan gang TT 1 }
        c1 := 0; tt := 2;
        { Phan con lai cua TT 1 }
Until false;

```

```

PAREND
END.

```

```

TT2: Repeat
    c2 := 1;
    While c1 = 1 do
        if tt = 1 then
            begin c2 := 0;
                while tt = 1 do ;
                    c2 := 1
            end;
        { Doan gang TT 2 }
        c2 := 0; tt := 1;
        { Phan con lai cua TT 2 }
Until false;

```

- **Ưu điểm:** Giải quyết được trường hợp tốc độ thực hiện của các tiến trình là khác nhau.
- **Nhược điểm:** Thuật toán trở nên vô cùng phức tạp khi số lượng tiến trình tăng.

Test and Set

Sơ đồ điều độ:

```

Var l1, l2, g: integer;
BEGIN
    g := 0;
    PARBEGIN

```

```

TT1: Repeat
    l1 := 1;
    While l1 = 1 do TS(l1); (đoạn vòng chờ)
    (Đoạn găng TT1)
    g := 0;
    (Phần còn lại của TT1)
Until false;

TT2: Repeat
    l2 := 1;
    While l2 = 1 do TS(l2);
    (Đoạn găng TT2)
    g := 0;
    (Phần còn lại của TT2)
Until false;
PAREND
End

```

Semaphore

- **Ý tưởng:** Mỗi tài nguyên găng được biểu diễn bởi 1 biến nguyên dương S . Đa số các tài nguyên găng đều có khả năng phục vụ $= 1$ ($S = 1$).
- Tồn tại 2 lệnh làm thay đổi giá trị của S :
 - $P(s) \rightarrow S - 1$. If $S < 0$ then đưa TT vào hàng đợi.
 - $V(s) \rightarrow S + 1$. If $S \leq 0$ then kích hoạt tiến trình đang xếp hàng.

Code mẫu:

```

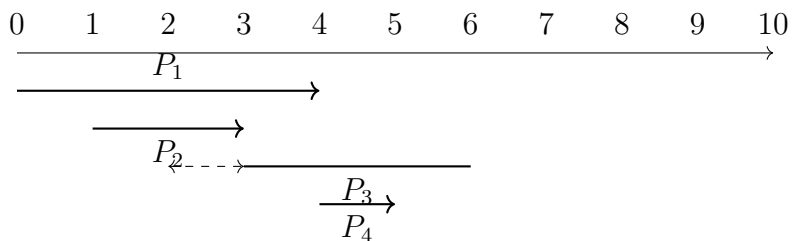
Var s: Integer
BEGIN
    s := 1;
    PARBEGIN
        TT1: Repeat
            P(s);
            (đoạn găng TT1)
            V(s);
            (phần còn lại của TT1)
            Until false;
        TT2: Repeat
            P(s);
            { đoạn găng TT2 }
            V(s);
            { phần còn lại của TT2 }
            Until false;
    PAREND
END.

```

Ví dụ $S = 2$:

Process	T.nhập	T.thực hiện
P_1	0	4
P_2	1	2
P_3	2	3
P_4	4	1

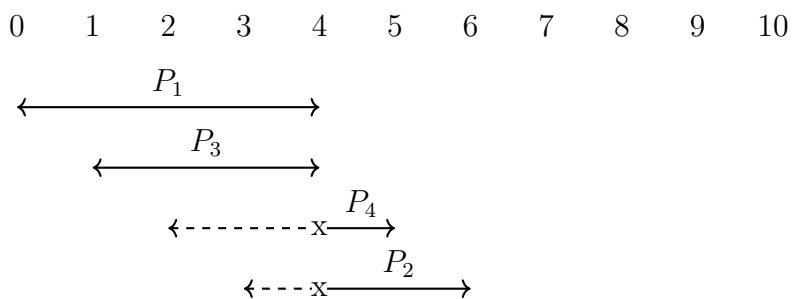
Sơ đồ hoạt động:



$S = 2$

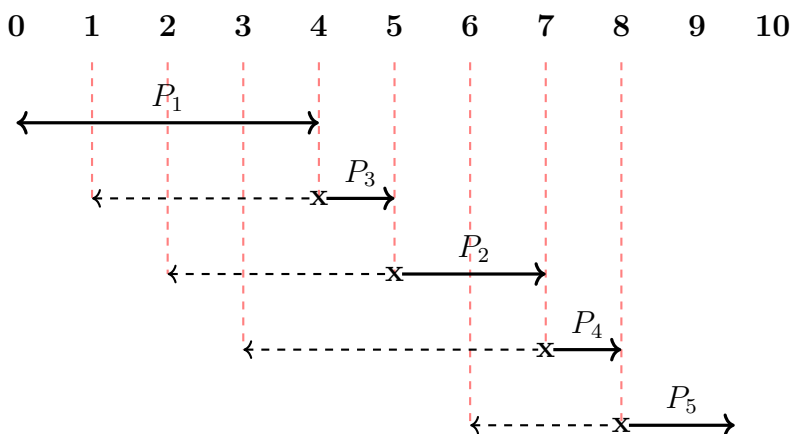
Process	T.nhập	T.thực hiện
P_1	0	4
P_2	3	2
P_3	1	3
P_4	2	1

Sơ đồ hoạt động:



Ví dụ $S = 1$ (Loại trừ tương hỗ/Mutex)

P	T.nhập	T.thực hiện
P_1	0	4
P_2	2	2
P_3	1	1
P_4	3	1
P_5	6	1



5. Bế tắc và xử lý bế tắc

- **Bế tắc:** Là trạng thái khi 2 hay nhiều tiến trình cùng chờ đợi một số sự kiện nào đó và nếu không có tác động từ bên ngoài thì sự chờ đợi đó là vô hạn.

Xét hệ thống có 12 tài nguyên là 12 băng từ và 3 tiến trình P_0, P_1, P_2 với các yêu cầu cấp phát:

- P_0 yêu cầu nhiều nhất 10 băng từ.
- P_1 yêu cầu nhiều nhất 4.
- P_2 yêu cầu nhiều nhất 9.

Giả sử tại thời điểm T_0 , P_0 đang chiếm 5, P_1 chiếm 2, P_2 chiếm 2. Như vậy còn $12 - (5 + 2 + 2) = 3$ băng từ rồi.

P	Max	Allocation	Need	Available
P_0	10	5	5	$3 - 1 = 2$
P_1	4	2	2	$2 + 2 = 4$
P_2	9	$2 + 1$	$7 - 1$	
		$= 3$	$= 6$	

$$\text{Need} = \text{Max} - \text{Allocation}$$

TO: $P_1 \rightarrow P_0 \rightarrow P_2$

T1: $P_1 \not\rightarrow$ Hệ thống k^o còn an toàn nữa

Available Initial: 3 3 2				
P	Allocation	Max	Avail (Trace)	Need
P_0	0 1 0	7 5 3	10 5 7	7 4 3
P_1	2 0 0	3 2 2	7 4 3	1 2 2
P_2	3 0 2	9 0 2	10 4 7	6 0 0
P_3	2 1 1	2 2 2	5 4 3	0 1 1
P_4	0 0 2	4 3 3	7 4 5	4 3 1

$$P_3 \rightarrow P_1 \rightarrow P_4 \rightarrow P_2 \rightarrow P_0$$

Chương 3: Quản lý bộ nhớ

Khái niệm:

- Bộ nhớ là 1 dãy các ô nhớ liên tiếp nhau.
- Bộ nhớ là thiết bị lưu trữ duy nhất mà qua đó CPU có thể thực hiện trao đổi thông tin với môi trường bên ngoài.
- HĐH có trách nhiệm cung cấp không gian nhớ cho các tiến trình khi có yêu cầu.

Phân mảnh ngoại (External Fragmentation):

- Kích thước không gian bộ nhớ còn trống đủ để thỏa mãn một yêu cầu cấp phát, tuy nhiên không gian nhớ này không liên tục $\Rightarrow$ phải dùng cơ chế kết nối.

Phân mảnh nội (Internal Fragmentation):

- Kích thước vùng nhớ được cấp phát lớn hơn vùng nhớ yêu cầu.
- Hiện tượng phân mảnh nội thường xảy ra khi bộ nhớ thực được chia thành các khối có kích thước cố định và các process được cấp phát theo đơn vị khối.

Quản lý bộ nhớ thật:

- Một process phải được nạp hoàn toàn vào bộ nhớ thì mới được thực thi.

Kỹ thuật phân vùng tĩnh nhiều hàng chờ:

- HĐH được load vào vùng nhớ thấp của RAM.
- Phần trống còn lại của RAM được chia thành nhiều phân vùng kích thước tăng dần (10K - 20K - 30K).
- Mỗi phân vùng có 1 hàng chờ tương ứng.
- Khi chạy ứng dụng HĐH sẽ chọn phân vùng có kích thước bé nhất nhưng $\geq$ kích thước của ứng dụng để sắp xếp vào hàng chờ tương ứng.
- HĐH sẽ phục vụ các tiến trình trong hàng chờ theo thuật toán FIFO.

Kỹ thuật phân vùng tĩnh 1 hàng chờ duy nhất:

- HĐH được load vào vùng nhớ thấp của RAM
- Phần trống còn lại của RAM được chia thành nhiều phân vùng kích thước tăng dần
- Chỉ có 1 hàng chờ duy nhất cho các ứng dụng.
- Khi 1 phân vùng trống, HĐH sẽ chọn 1 tiến trình trong hàng chờ để phục vụ.

Các chiến lược cấp phát bộ nhớ:

1. **First-Fit:** Cấp khối nhớ đầu tiên thỏa mãn.
2. **Best-Fit:** Cấp phát bộ nhớ bé nhất đủ để đáp ứng cho tiến trình. Phải duyệt toàn bộ danh sách.
3. **Worst-Fit:** Cấp phát bộ nhớ lớn nhất đủ để đáp ứng cho tiến trình. Phải duyệt toàn bộ danh sách.

$\rightarrow$ Như vậy: First-Fit và Best-Fit tốt hơn Worst-Fit về vấn đề tốc độ và tận dụng bộ nhớ.

BTVN: Có các phân vùng nhớ: $S_0(80K)$, $S_1(220K)$, $S_2(290K)$, $S_3(380K)$, $S_4(580K)$, $S_5(620K)$.

Các tiến trình: $P_0(110K)$, $P_1(200K)$, $P_2(550K)$, $P_3(500K)$, $P_4(610K)$.

Kỹ thuật phân vùng động (tiêu đề).

Quản lý bộ nhớ ảo

- Đơn vị quản lý bộ nhớ ảo MMU.

Nguyên lý:

- HĐH chia bộ nhớ ảo của chương trình thành nhiều đơn vị có kích thước bằng nhau, mỗi đơn vị này là 1 trang (page). Kích thước trang ảo (256, 512, 1K, 2K ...).
- Bộ nhớ RAM cũng được chia thành nhiều đơn vị quản lý, mỗi đơn vị được gọi là khung trang thật (frame) và có kích thước = trang ảo.